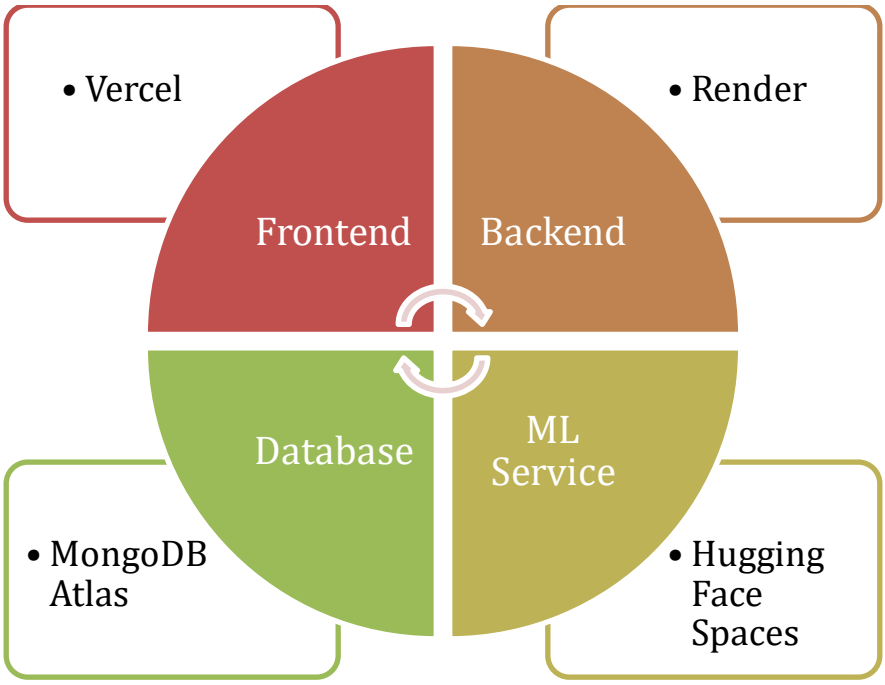
KNEEVISION

Deployment Manual

AI-Assisted Knee X-Ray Analysis Platform



1. Overview

KneeVision is a full-stack AI-assisted knee X-ray analysis platform built as a STARLABS capstone project at Pace University. This document covers both production cloud deployment and local Docker-based deployment.

1.1 Architecture

The system comprises four services communicating over HTTPS in production:

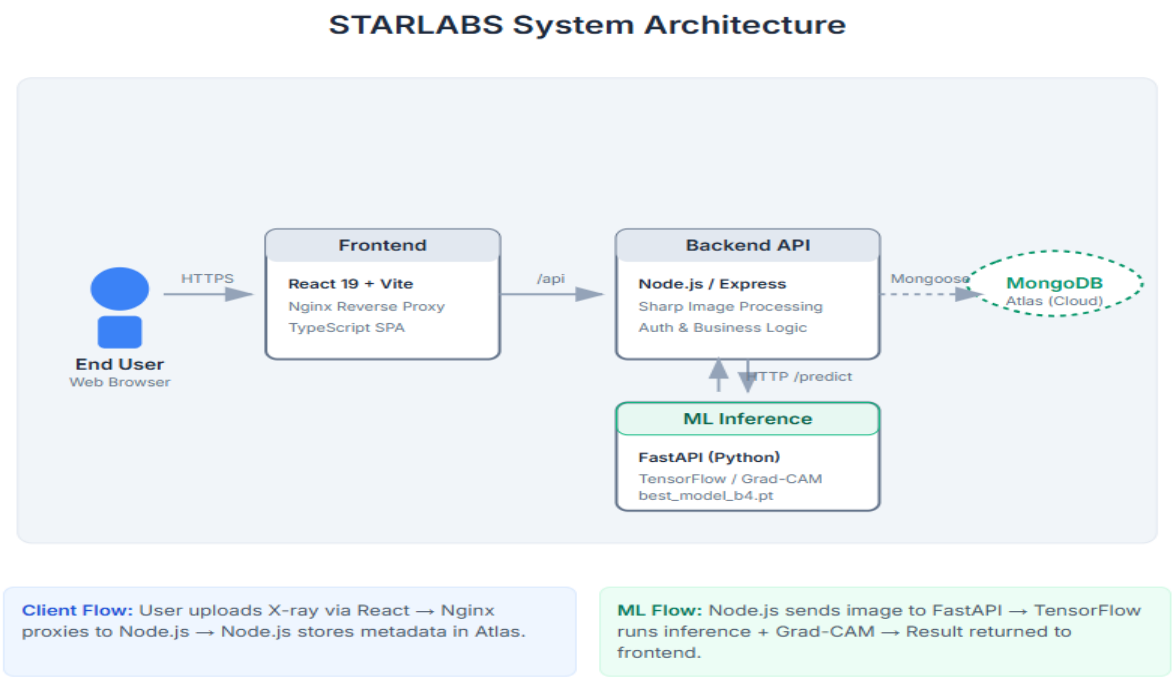


Figure 1.1: KneeVision production architecture

Service	Technology
Frontend	React 19 + Vite + TypeScript, served via Vercel CDN
Backend	Node.js 20 + Express 5, deployed on Render
ML Service	Python FastAPI + TensorFlow, deployed on Hugging Face Spaces
Database	MongoDB Atlas M0 free cluster (512 MB)
Model file	model.hdf5 hosted on Hugging Face Model Hub (not in Git)

1.2 Repository

The codebase is hosted on a shared GitHub repository provided by the professor. The repository structure is:

```
STARLABS/
├── backend/          Node.js Express API
├── frontend/         React + Vite
├── ml-service/       FastAPI + TensorFlow inference service
└── infra/            Docker Compose for local development
```

Important: Model File

The model file (best_model_b4.pt) is NOT committed to Git — it is too large for Git.

LOCAL DEV: Download it from the Google Drive link inside ml-service/README.md on GitHub.

PRODUCTION: The model is stored in a PRIVATE Hugging Face Model Hub repository.

The HF Space pulls it automatically at startup using a secret token (HUGGINGFACE_TOKEN).

The private HF repo URL is never shared or exposed publicly.

2. Prerequisites

2.1 Accounts Required

Platform	Notes
GitHub	Access to the shared class repository
Vercel	vercel.com — free Hobby plan
Render	render.com — free tier
Hugging Face	huggingface.co — free account
MongoDB Atlas	mongodb.com/atlas — free M0 cluster

2.2 Local Tools (for local deployment only)

- Docker Desktop 4.x or later
- Node.js 20+ and npm
- Git
- Vercel CLI: npm install -g vercel

2.3 Clone the Repository

Clone the shared repository provided by your professor:

```
git clone <PROFESSOR_GITHUB_REPO_URL>
cd STARLABS
```

3. ML Service — Hugging Face Spaces

The ML Service (FastAPI + TensorFlow) is deployed as a Docker Space on Hugging Face. There are two separate concerns: (1) getting the model file for local development and (2) deploying to production where the model is private.

3.1 Get the Model File (Local Development)

The model file is not in Git. For local development, download it from the Google Drive link published in the ml-service/README.md file inside the GitHub repository.

1 Open the GitHub repository and navigate to ml-service/README.md

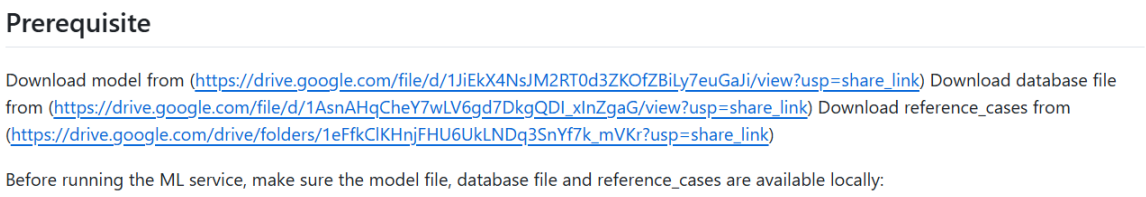


Figure 3.1: The Google Drive link is in ml-service/README.md on GitHub

2 Click the Google Drive link in the README and download the model file

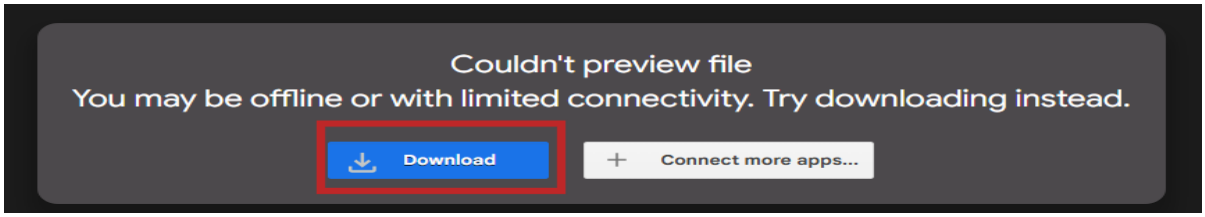


Figure 3.2: Download best_model_b4.pt from Google Drive

3 Place the downloaded file at: ml-service/best_model_b4.pt

```
# Your folder should look like this after downloading:
ml-service/
├── reference_cases/
├── app.py
├── Dockerfile
├── requirements.txt
├── kneevision_final.db
├── README.md          ← contains the Google Drive link
└── best_model_b4.pt   ← place the downloaded file here
```

Local only — never commit the model

The model file is bind-mounted into the Docker container at runtime (not baked into the image).

Never add best_model_b4.pt to Git — it is listed in .gitignore for this reason.

The Google Drive link is the only way to get the file for local development.

3.2 Production Model — Private Hugging Face Repository

For production, the model is stored in a private Hugging Face Model Hub repository. The private repo URL is never shared. The HF Space pulls the model automatically at container startup using a secret access token.

Security note

The model repository on Hugging Face is set to PRIVATE. This means:

- No public URL is ever exposed in code, documentation, or environment variables.
- Only a Hugging Face access token with read permission can download the model.
- The token is stored as a Space secret — never in the codebase or git history.
- Deployers do NOT need to know the private repo URL. The app.py already has it.

The ml-service/app.py is pre-configured with the private repo ID and downloads the model on startup:

```
from huggingface_hub import hf_hub_download
import os

# Downloads from the private HF repo using the token in the environment
# The repo ID is already set in app.py — deployers only need to supply the token
MODEL_PATH = hf_hub_download(
    repo_id='TEAM_HF_USERNAME/kneevision-model',  # private repo, pre-set
    filename='best_model_b4.pt',
    token=os.environ['HUGGINGFACE_TOKEN']  # injected as Space secret
)
```

3.3 Create a Hugging Face Space

- Log in to huggingface.co and click New → Space
- Configure the Space as follows:

Setting	Value
Space name	kneevision-ml (or your preferred name)
SDK	Docker
Visibility	Public (the Space is public; the model repo is private)

Space vs Model repo visibility

The SPACE (the running app) is PUBLIC — this is required for the free tier and is fine.

The MODEL REPO (where best_model_b4.pt is stored) is PRIVATE — it stays secret.

The Space accesses the private model repo via the HUGGINGFACE_TOKEN secret.

Nobody can reach the model file directly, even though the Space URL is public.

- Push the ml-service folder contents to the Space repository

The Space is its own Git repo. Push only the ml-service/ files — not the whole monorepo:

```
# Clone your new Space repo
git clone https://huggingface.co/spaces/YOUR_HF_USERNAME/kneevision-ml
cd kneevision-ml

# Copy ml-service contents into the Space repo
cp -r ../STARLABS/ml-service/* .

# Do NOT copy the model file — it is downloaded at runtime
# Confirm .gitignore excludes *.pt and *.hdf5

git add .
git commit -m "Initial deployment"
git push
```

3.4 Set Space Secrets (Required)

The HUGGINGFACE_TOKEN secret is the only credential needed. It grants the container read access to the private model repository. Without it, the container will fail to start.

- In your Space, go to Settings → Variables and Secrets

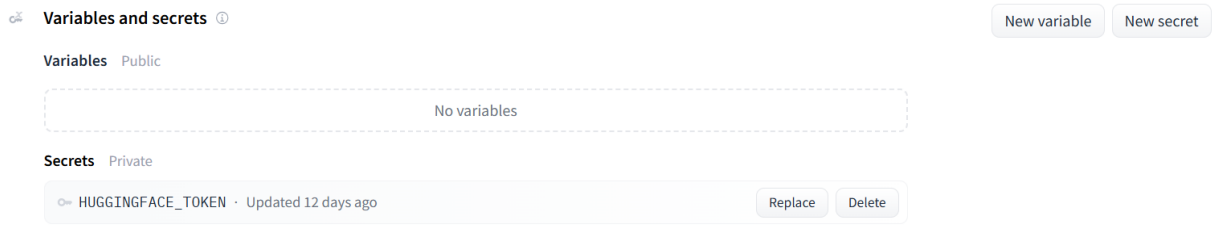


Figure 3.3: Add HUGGINGFACE_TOKEN as a Space secret — it will be masked after saving

Generate a read-only token at huggingface.co/settings/tokens. Use a Fine-grained token with Read access to the private model repo only:

Secret / Variable	Value
HUGGINGFACE_TOKEN	Your HF read token — generated at hf.co/settings/tokens
LAST_CONV_LAYER_NAME	global_average_pooling2d

Token type recommendation
Use a Fine-grained token (not a legacy token) scoped to read-only access on the private model repo.
This limits the blast radius if the token is ever accidentally exposed.
Regenerate the token if you suspect it has been compromised.

3.5 Confirm Deployment

After pushing, Hugging Face builds the Docker image and starts the container. The first startup takes longer than usual because the model file (several hundred MB) is downloaded from the private repo.

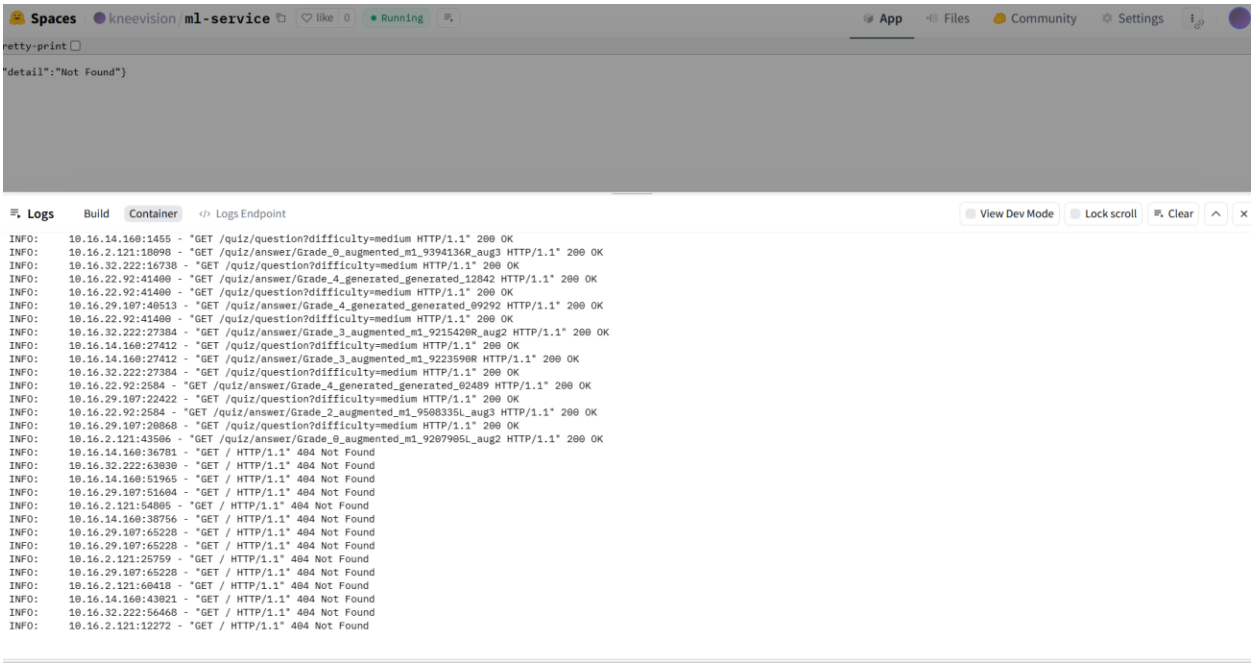


Figure 3.4: Space logs showing model download and successful startup

Your ML endpoint once running:

```
https://YOUR_HF_USERNAME-kneevision-ml.hf.space
```

Test it:

```
curl https://YOUR_HF_USERNAME-kneevision-ml.hf.space/health
```

4. MongoDB Atlas — Free Cluster

1 Go to mongodb.com/atlas and sign in or create a free account

2 Create a new project and deploy a free M0 cluster

3 Under Security → Database Access, create a database user

- Username: kneevision
- Password: generate a strong password — save it
- Role: Atlas admin (or readWriteAnyDatabase)

4 Under Security → Network Access, add IP address

- For Render backend: click Add IP Address → Allow Access from Anywhere (0.0.0.0/0)

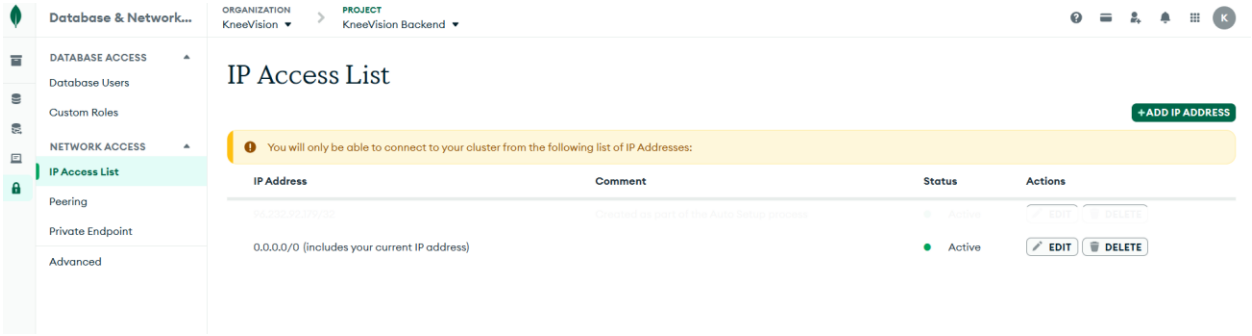


Figure 4.1: Allow access from anywhere for Render compatibility

5 Get your connection string

Go to your cluster → Connect → Drivers → copy the connection string:

```
mongodb+srv://kneevision:<PASSWORD>@cluster0.xxxxx.mongodb.net/kneevision?retryWrites=true&w=majority
```

Replace <PASSWORD> with your actual password. Save this string — you will need it for Render.

5. Backend — Render

The Node.js backend is deployed as a Web Service on Render. Point Render directly at the shared GitHub repository and specify the backend subfolder.

5.1 Create the Web Service

1 Go to render.com, sign in, and click New → Web Service

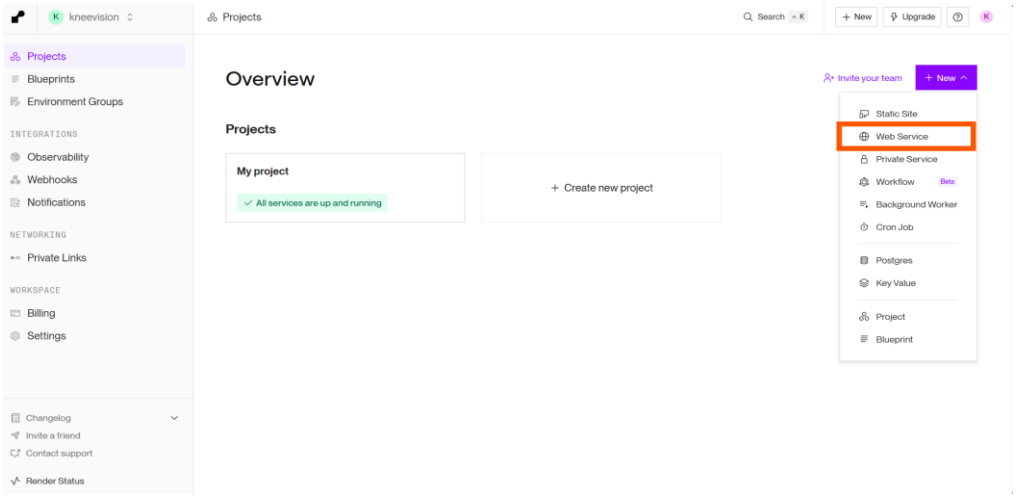


Figure 5.1: Create a new Web Service on Render

2 Connect your GitHub account and select the professor's shared repository

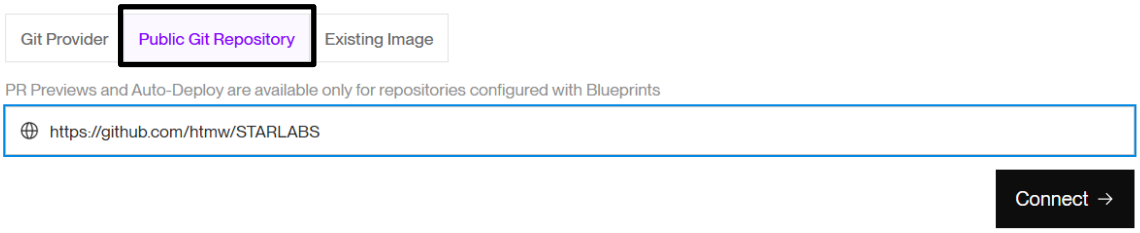


Figure 5.2: Connect the shared GitHub repository

3 Configure the service settings as follows:

Setting	Value
Name	kneevision-backend (or any name)
Region	US East (Ohio) — or closest to you
Branch	main
Root Directory	backend
Runtime	Node
Build Command	npm install
Start Command	node src/index.js
Instance Type	Free

Root Directory

Setting Root Directory to 'backend' tells Render to treat the backend/ subfolder as the project root. This means Render will run npm install inside backend/, not the repo root.

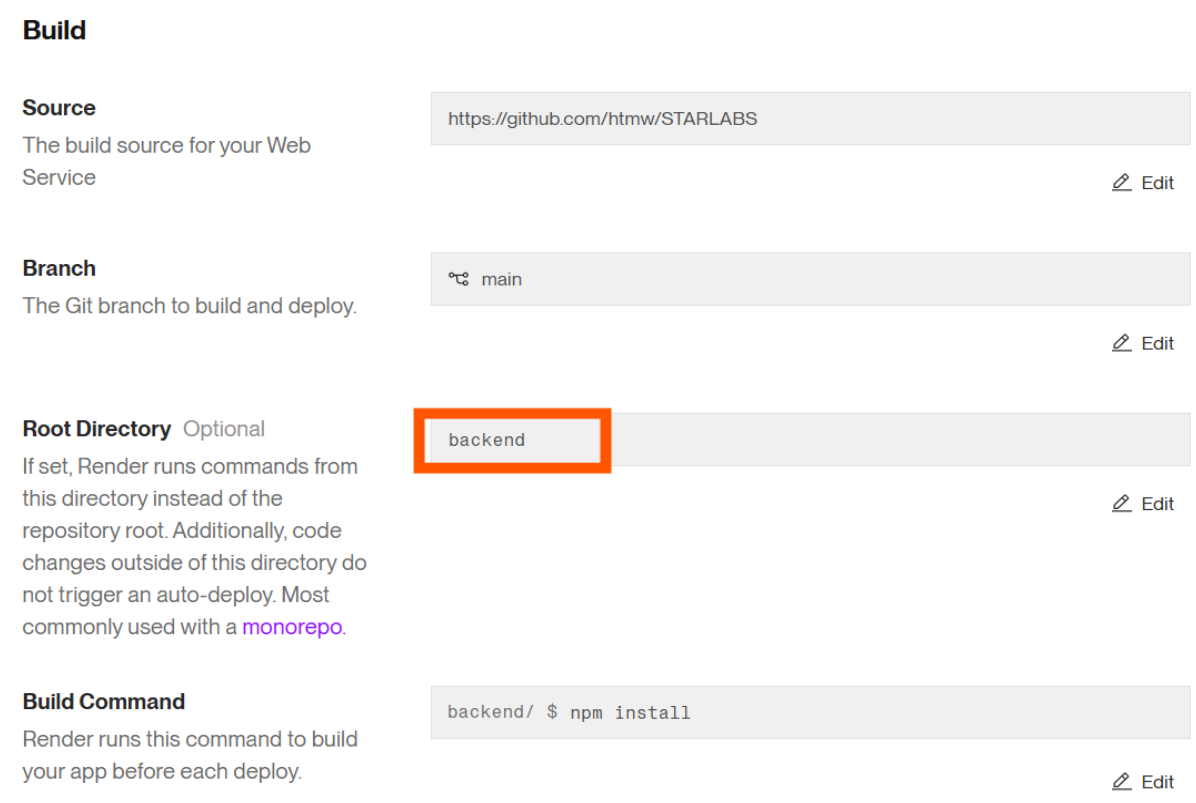


Figure 5.3: Service configuration — note the Root Directory field

5.2 Set Environment Variables

In Render's service page, go to Environment and add the following variables:

Variable	Value
MONGODB_URI	Your MongoDB Atlas connection string from Section 4
JWT_SECRET	A strong random string — generate with: openssl rand -hex 32
PORT	4000
ML_SERVICE_URL	Your Hugging Face Space URL from Section 3.4
GROQ_API_KEY	Your Groq API key from console.groq.com
NODE_ENV	production

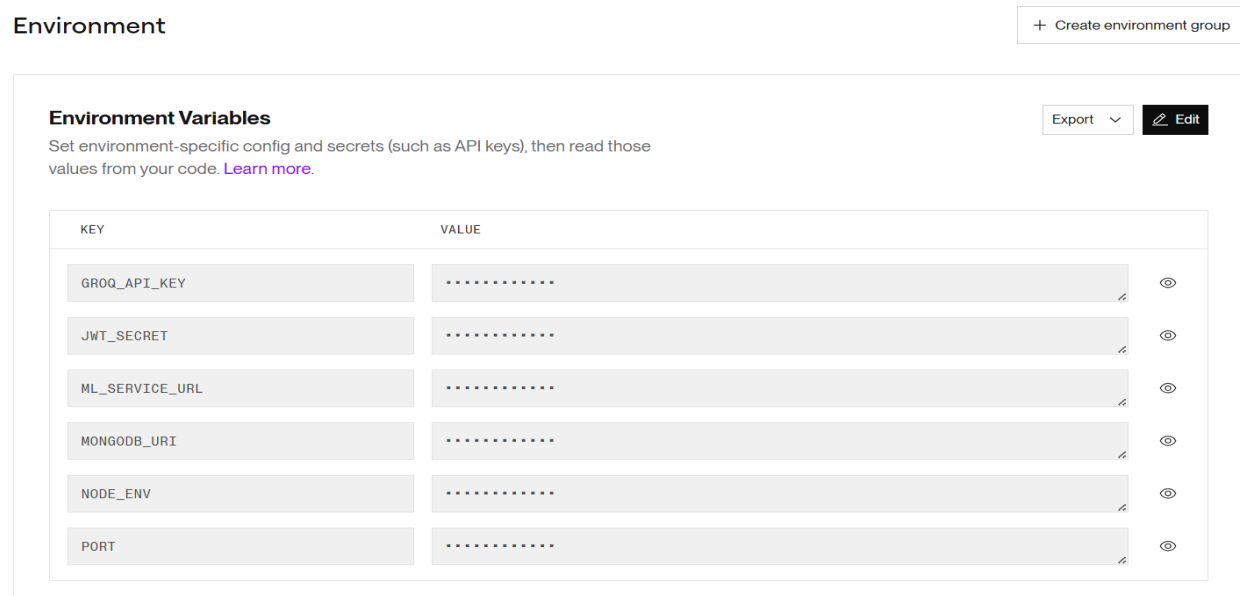


Figure 5.4: All environment variables set in Render

4 Click Create Web Service. Render will build and deploy automatically.

Your backend endpoint will be:

```
https://kneevision-backend.onrender.com
```

Free Tier Note

Render's free tier spins down web services after 15 minutes of inactivity.

The first request after idle takes ~30 seconds to wake up. This is normal for a demo.

Upgrade to the Starter plan ($7/month) to eliminate cold starts.

5.3 Verify the Backend

Once deployed, test the health endpoint:

```
curl https://kneevision-backend.onrender.com/health
```

Expected response:

```
{"status":"ok"}
```

6. Frontend — Vercel

The frontend is deployed to Vercel using the Vercel CLI. Since the codebase is in a shared GitHub repository you may not own, we use manual CLI deployment rather than GitHub integration.

6.1 Install the Vercel CLI

```
npm install -g vercel
```

6.2 Log in to Vercel

```
vercel login
```

Follow the browser prompt to authenticate with your Vercel account.

6.3 Deploy the Frontend

- Navigate to the frontend folder in the cloned repository:

```
cd STARLABS/frontend
```

- Run the Vercel deployment command:

```
npx vercel
```

- Answer the Vercel CLI prompts:

Prompt	Answer
Set up and deploy?	Y
Which scope?	Your personal Vercel account
Link to existing project?	N (first deploy)
Project name?	kneevision (or any name)
In which directory is your code?	Frontend/
Want to override the settings?	N

Figure 6.1: Vercel CLI deployment — note the production URL

6.4 Set the Environment Variable

After the first deploy, set the backend URL. This is a Vite build-time variable and requires a redeploy after setting:

```
# Set the env var in Vercel
vercel env add VITE_API_BASE_URL production
# When prompted, enter your Render backend URL:
# https://kneevision-backend.onrender.com

# Redeploy to pick up the variable
vercel --prod
```

Alternatively, set it in the Vercel dashboard under Project → Settings → Environment Variables:

Variable	Value
VITE_API_BASE_URL	https://kneevision-backend.onrender.com

6.5 Verify the Deployment

Open the Vercel URL in your browser. You should see the KneeVision landing page. Try registering a new account and uploading an X-ray.

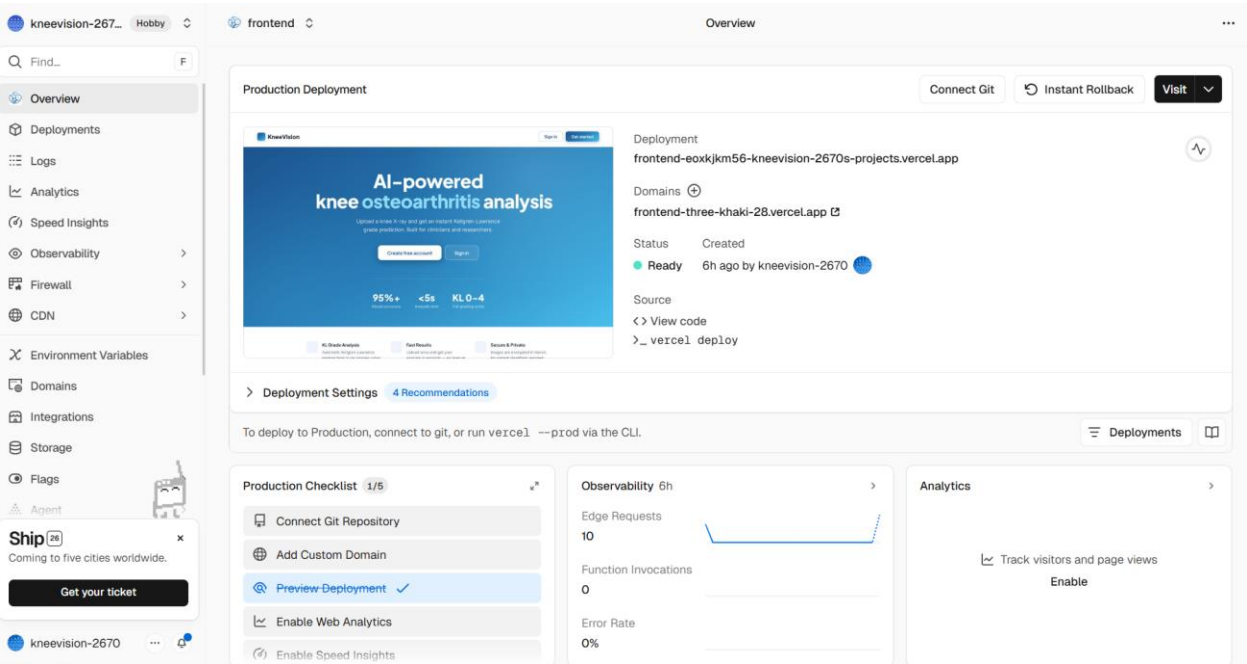


Figure 6.2: KneeVision running in production

7. Local Deployment — Docker Compose

For local development and testing, all services run inside Docker containers using Docker Compose. Hot-reload for frontend and backend is also available without Docker.

7.1 Prerequisites

- Docker Desktop installed and running
- The model file (best_model_b4.pt) placed at: ml-service/best_model_b4.pt
- A MongoDB Atlas connection string (or use the local mongo container below)

7.2 Configure Environment Variables

The infra/ folder contains the docker-compose.yml. Create a .env file in the infra/ folder:

```
cd STARLABS/infra
cp .env.example .env
```

Edit the .env file with the following values:

```
# ── Backend ──────────────────────────────────────────────
# Use MongoDB Atlas (recommended):
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/kneevision

JWT_SECRET=dev-secret-change-me
PORT=4000

# ML service inside Docker network:
ML_SERVICE_URL=http://ml-service:8000

# ── ML Service ───────────────────────────────────────────
HUGGINGFACE_TOKEN=YOUR_HUGGINGFACE_TOKEN
LAST_CONV_LAYER_NAME=global_average_pooling2d

# ── Frontend ─────────────────────────────────────────────
VITE_BACKEND_URL=http://localhost:4000

# ── Groq ─────────────────────────────────────────────────
GROQ_API_KEY=YOUR_GROQ_API_KEY
```

MongoDB for Local Dev

The docker-compose.yml includes a mongo service with profiles: [local].

To use the local MongoDB instead of Atlas, add --profile local to the Docker Compose command and set MONGODB_URI=mongodb://mongo:27017/kneevision in your .env file.

7.3 Docker Compose File

The infra/docker-compose.yml orchestrates all four services:

Service	Description
mongo	MongoDB 7 (local dev only, profiles: local)
backend	Node.js Express on port 4000
frontend	React + nginx on port 80
ml-service	FastAPI on port 8000, model bind-mounted

7.4 Start All Services

```
cd STARLABS/infra

# Production-mode Docker (uses Atlas MongoDB):
docker compose up --build

# Local mode with local MongoDB:
docker compose --profile local up --build
```

The first build takes several minutes as Docker pulls base images and installs dependencies. Subsequent starts are fast due to layer caching.

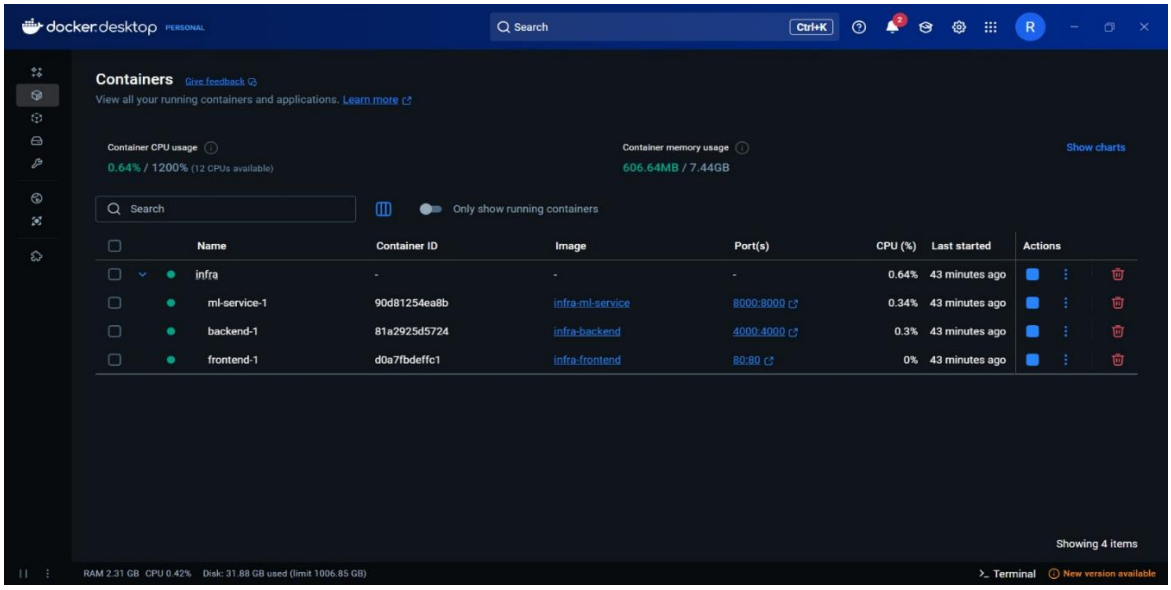


Figure 7.1: All Docker services running

Service	Local URL
Frontend	http://localhost
Backend API	http://localhost:4000
ML Service	http://localhost:8000
MongoDB (local)	mongodb://localhost:27017

7.5 Hot Reload (without Docker)

For fast development without rebuilding Docker images, run the frontend and backend directly with hot reload:

Backend

```
cd STARLABS/backend
npm install
npm run dev     # nodemon watches src/ and reloads on changes
```

Frontend

```
cd STARLABS/frontend
npm install
npm run dev     # Vite HMR at http://localhost:5173
```

While using hot reload, set VITE_BACKEND_URL=http://localhost:4000 in frontend/.env.local and ensure the backend is running.

8. End-to-End Smoke Test

Run through the following checklist after any deployment to confirm all services are connected and working:

Check	Expected behavior
Frontend loads	Open the Vercel URL (or localhost). Landing page renders without errors.
Register a user	Create a new account. Should succeed and redirect to the upload page.
Login	Log out and log back in. JWT token should persist.
Upload an X-ray	Upload a knee X-ray JPEG or PNG. Image should appear in the gallery.
Run prediction	Click Analyze. The ML service should return a KL grade and confidence.
View result	Result page shows grade, confidence bar, Grad-CAM heatmap.
History tab	Navigate to Analysis History. Past predictions should be listed.
Re-view result	Click View Result on a history item. Result page re-renders from saved data.

Render cold start
If the first prediction is slow (30+ seconds), the Render backend woke from idle.
Subsequent requests will be fast. This is normal on the free tier.

9. Troubleshooting

Symptom	Fix
Render shows 'Service Unavailable'	Check Render Logs tab. Ensure MONGODB_URI and JWT_SECRET are set correctly.
ML prediction timeout	Hugging Face Spaces free tier has cold starts. Wait 60s and retry. Check Space logs.
CORS error in browser	Ensure VITE_API_BASE_URL points to the Render backend URL, not localhost.
Model not found on startup	HUGGINGFACE_TOKEN is missing or invalid in Space secrets. Regenerate token.
MongoDB authentication failed	Password in connection string has special characters. URL-encode them.
Frontend shows blank page	Open browser DevTools → Console. Usually a missing env var. Redeploy after setting.
Docker compose port conflict	Port 80 or 4000 in use. Change the left side of the port mapping in docker-compose.yml.
npm run dev fails	Run npm install first inside the specific service folder (backend/ or frontend/).

10. Quick Reference

Production URLs (fill in after deployment)

Service	URL
Frontend (Vercel)	https://kneevision.vercel.app
Backend (Render)	https://kneevision-backend.onrender.com
ML Service (HF Spaces)	https://USERNAME-kneevision-ml.hf.space
MongoDB Atlas	(connection string, not a URL)

All Environment Variables at a Glance

Variable	Where it is set
MONGODB_URI	Backend (Render + local Docker)
JWT_SECRET	Backend (Render + local Docker)
PORT	Backend — set to 4000
ML_SERVICE_URL	Backend — Hugging Face Space URL
GROQ_API_KEY	Backend — from console.groq.com
NODE_ENV	Backend — set to production
HUGGINGFACE_TOKEN	ML Service (Space secret + local Docker)
LAST_CONV_LAYER_NAME	ML Service — global_average_pooling2d
VITE_API_BASE_URL	Frontend build-time (Vercel + local)
VITE_BACKEND_URL	Frontend local dev only

Useful Commands

```
# Rebuild and restart all Docker services
docker compose up --build
# View logs for a specific service
docker compose logs -f backend
docker compose logs -f ml-service

# Stop all services
docker compose down

# Redeploy frontend to Vercel
cd frontend && vercel --prod

# Test ML health
curl https://USERNAME-kneevision-ml.hf.space/health

# Test backend health
curl https://kneevision-backend.onrender.com/health
```